1. 模拟实现 std::array

```cpp
#include <iostream>
#include <initializer_list>
#include <utility>    // for std::move
#include <stdexcept>  // for std::out_of_range
#include <memory>     // for std::unique_ptr

//============================================================
// my_array_raw: 使用手动内存管理的版本
// 模仿 std::array 接口，但内部使用 new/delete 管理固定大小的数组。
//============================================================

template<typename T, std::size_t N>
class my_array_raw {
private:
    T* data_; // 原始指针，指向动态分配的内存块，大小为 N 个 T 元素

public:
    // (1) 默认构造函数：为 N 个元素分配内存，并默认构造它们
    // 此处假设 T 有默认构造函数，如需更严格处理可加上 static_assert 检查。
    my_array_raw() : data_(new T[N]()) {
        // 这里使用了值初始化，即对于内置类型会初始化为 0，对于类类型会调用默认构造。
    }

    // (2) 从初始化列表构造：允许使用类似 std::array 的初始化方式
    // 如: my_array_raw<int, 3> arr = {1, 2, 3};
    my_array_raw(std::initializer_list<T> init) : data_(new T[N]()) {
        if (init.size() > N) {
            throw std::out_of_range("Initializer list size exceeds my_array capacity");
        }
        // 将初始化列表中的元素拷贝到 data_
        std::size_t i = 0;
        for (const auto& elem : init) {
            data_[i++] = elem;
        }
        // 若 init.size() < N，剩余元素已通过 new T[N]() 初始化为默认值
    }

    // (3) 拷贝构造函数：深拷贝，分配新内存并逐元素拷贝
    my_array_raw(const my_array_raw& other) : data_(new T[N]) {
        for (std::size_t i = 0; i < N; ++i) {
            data_[i] = other.data_[i];
        }
    }

    // (4) 移动构造函数：转移所有权，避免无谓的拷贝
```

```cpp
// 移动后other.data_应指向nullptr，避免双重释放
my_array_raw(my_array_raw&& other) noexcept : data_(other.data_) {
    other.data_ = nullptr; // 将other 的指针置空，表示资源已移动
}

// (5) 拷贝赋值运算符：深拷贝
// 先自我赋值检查，再拷贝，确保安全性
my_array_raw& operator=(const my_array_raw& other) {
    if (this == &other) return *this; // 自我赋值检查
    T* new_data = new T[N]; // 分配临时内存防止异常中途导致资源泄漏
    for (std::size_t i = 0; i < N; ++i) {
        new_data[i] = other.data_[i];
    }
    delete[] data_;  // 释放旧资源
    data_ = new_data; // 接管新资源
    return *this;
}

// (6) 移动赋值运算符：转移所有权
// 删除旧内存，接受other 的data_指针，并令other 置空
my_array_raw& operator=(my_array_raw&& other) noexcept {
    if (this == &other) return *this;
    delete[] data_;
    data_ = other.data_;
    other.data_ = nullptr;
    return *this;
}

// (7) 析构函数：释放分配的内存
~my_array_raw() {
    delete[] data_;
}

// 返回数组大小，与std::array 一致
constexpr std::size_t size() const noexcept {
    return N;
}

// 下标运算符，无边界检查
T& operator[](std::size_t index) noexcept {
    return data_[index];
}

const T& operator[](std::size_t index) const noexcept {
    return data_[index];
}
```

```
    // at 成员函数，带边界检查，超出范围则抛出异常
    T& at(std::size_t index) {
        if (index >= N) {
            throw std::out_of_range("Index out of range in my_array_raw::at()");
        }
        return data_[index];
    }

    const T& at(std::size_t index) const {
        if (index >= N) {
            throw std::out_of_range("Index out of range in my_array_raw::at()");
        }
        return data_[index];
    }
};

//============================================================
// my_array_smart: 使用智能指针进行内存管理的版本
// 同样模仿 std::array 接口，但内部使用 std::unique_ptr<T[]> 来管理内存。
// 这可以免除手动 delete[] 的麻烦，并更好地适应异常安全。
//============================================================

template<typename T, std::size_t N>
class my_array_smart {
private:
    // 使用 unique_ptr 管理动态分配的数组，当 my_array_smart 对象析构时 unique_ptr 会自动释放
    std::unique_ptr<T[]> data_;

public:
    // (1) 默认构造函数
    my_array_smart() : data_(std::make_unique<T[]>(N)) {
        // std::make_unique<T[]>(N) 分配 N 个默认构造的 T 元素
    }

    // (2) 从初始化列表构造
    my_array_smart(std::initializer_list<T> init) : data_(std::make_unique<T[]>(N)) {
        if (init.size() > N) {
            throw std::out_of_range("Initializer list size exceeds my_array_smart capac
ity");
        }
        std::size_t i = 0;
        for (const auto& elem : init) {
            data_[i++] = elem;
        }
    }
```

```cpp
// (3) 拷贝构造函数：深拷贝
// 虽然unique_ptr 不支持拷贝，但这里我们通过新分配一块内存手动拷贝数据实现深拷贝
my_array_smart(const my_array_smart& other) : data_(std::make_unique<T[]>(N)) {
    for (std::size_t i = 0; i < N; ++i) {
        data_[i] = other.data_[i];
    }
}

// (4) 移动构造函数：std::unique_ptr 支持移动语义，直接 move 即可
my_array_smart(my_array_smart&& other) noexcept : data_(std::move(other.data_)) {
    // 移动后 other.data_ 会变为 nullptr，资源安全转移
}

// (5) 拷贝赋值运算符：深拷贝
my_array_smart& operator=(const my_array_smart& other) {
    if (this == &other) return *this;
    // 创建新内存并拷贝数据
    auto new_data = std::make_unique<T[]>(N);
    for (std::size_t i = 0; i < N; ++i) {
        new_data[i] = other.data_[i];
    }
    data_ = std::move(new_data);
    return *this;
}

// (6) 移动赋值运算符：利用 unique_ptr 的移动赋值即可
my_array_smart& operator=(my_array_smart&& other) noexcept {
    if (this == &other) return *this;
    data_ = std::move(other.data_);
    return *this;
}

// (7) 析构函数：unique_ptr 自动处理内存释放，无需显式操作
~my_array_smart() = default;

// 大小函数
constexpr std::size_t size() const noexcept {
    return N;
}

// 下标操作符
T& operator[](std::size_t index) noexcept {
    return data_[index];
}

const T& operator[](std::size_t index) const noexcept {
    return data_[index];
```

```
    }

    // 带边界检查的at 函数
    T& at(std::size_t index) {
        if (index >= N) {
            throw std::out_of_range("Index out of range in my_array_smart::at()");
        }
        return data_[index];
    }

    const T& at(std::size_t index) const {
        if (index >= N) {
            throw std::out_of_range("Index out of range in my_array_smart::at()");
        }
        return data_[index];
    }
};

//============================================================
// 测试示例
//============================================================
int main() {
    // 使用my_array_raw
    my_array_raw<int, 5> arr1 = {1, 2, 3, 4, 5};
    // 测试访问
    for (std::size_t i = 0; i < arr1.size(); ++i) {
        std::cout << "arr1[" << i << "] = " << arr1[i] << "\n";
    }

    // 使用my_array_smart
    my_array_smart<double, 3> arr2 = {3.14, 2.71, 1.41};
    // 测试at 访问
    for (std::size_t i = 0; i < arr2.size(); ++i) {
        std::cout << "arr2.at(" << i << ") = " << arr2.at(i) << "\n";
    }

    // 测试拷贝与移动
    my_array_raw<int, 5> arr3 = arr1; // 拷贝构造
    my_array_raw<int, 5> arr4 = std::move(arr3); // 移动构造

    my_array_smart<double, 3> arr5 = arr2; // 拷贝构造
    my_array_smart<double, 3> arr6 = std::move(arr5); // 移动构造

    return 0;
}
```

2. 模拟实现 std::vector

```cpp
#include <iostream>
#include <initializer_list>
#include <utility>      // for std::move
#include <stdexcept>    // for std::out_of_range
#include <memory>       // for std::unique_ptr
#include <algorithm>    // for std::copy, std::move

//============================================================
// my_vector_raw: 使用手动内存管理的版本
// 模仿 std::vector 接口，内部使用 new/delete 管理动态数组。
//============================================================

template<typename T>
class my_vector_raw {
private:
    T* data_;              // 指向动态分配的数组
    std::size_t size_;     // 当前元素数量
    std::size_t capacity_; // 当前容量

    // 内部函数：扩展容量到新的容量
    void reserve(std::size_t new_capacity) {
        if (new_capacity <= capacity_) return;
        // 分配新内存
        T* new_data = new T[new_capacity];
        // 拷贝现有元素到新内存
        for (std::size_t i = 0; i < size_; ++i) {
            new_data[i] = std::move(data_[i]);
        }
        // 释放旧内存
        delete[] data_;
        data_ = new_data;
        capacity_ = new_capacity;
    }

public:
    // (1) 默认构造函数
    my_vector_raw() : data_(nullptr), size_(0), capacity_(0) {}

    // (2) 从初始化列表构造
    my_vector_raw(std::initializer_list<T> init) : data_(nullptr), size_(0), capacity_(
0) {
        reserve(init.size());
        for (const auto& elem : init) {
            push_back(elem);
        }
    }
```

```
    // (3) 拷贝构造函数：深拷贝
    my_vector_raw(const my_vector_raw& other) : data_(nullptr), size_(0), capacity_(0)
{
        reserve(other.capacity_);
        for (std::size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
    }

    // (4) 移动构造函数：转移所有权
    my_vector_raw(my_vector_raw&& other) noexcept
    : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
        other.data_ = nullptr;
        other.size_ = 0;
        other.capacity_ = 0;
    }

    // (5) 拷贝赋值运算符：深拷贝
    my_vector_raw& operator=(const my_vector_raw& other) {
        if (this == &other) return *this; // 自我赋值检查
        // 清理当前资源
        delete[] data_;
        data_ = nullptr;
        size_ = 0;
        capacity_ = 0;
        // 拷贝新数据
        reserve(other.capacity_);
        for (std::size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
        return *this;
    }

    // (6) 移动赋值运算符：转移所有权
    my_vector_raw& operator=(my_vector_raw&& other) noexcept {
        if (this == &other) return *this; // 自我赋值检查
        // 释放当前资源
        delete[] data_;
        // 转移资源
        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;
        // 重置 other
        other.data_ = nullptr;
        other.size_ = 0;
        other.capacity_ = 0;
```

```
    return *this;
}

// (7) 析构函数：释放动态内存
~my_vector_raw() {
    delete[] data_;
}

// 添加元素到末尾
void push_back(const T& value) {
    if (size_ == capacity_) {
        // 增加容量，通常是现有容量的两倍，或至少为1
        std::size_t new_capacity = capacity_ == 0 ? 1 : capacity_ * 2;
        reserve(new_capacity);
    }
    data_[size_++] = value;
}

// 移除末尾元素
void pop_back() {
    if (size_ == 0) {
        throw std::out_of_range("pop_back() called on empty my_vector_raw");
    }
    --size_;
    // 可选：调用析构函数（对于非平凡类型）
    // data_[size_].~T();
}

// 返回当前大小
std::size_t size() const noexcept {
    return size_;
}

// 返回当前容量
std::size_t capacity() const noexcept {
    return capacity_;
}

// 检查是否为空
bool empty() const noexcept {
    return size_ == 0;
}

// 下标操作符，无边界检查
T& operator[](std::size_t index) noexcept {
    return data_[index];
}
```

```
    const T& operator[](std::size_t index) const noexcept {
        return data_[index];
    }

    // at 成员函数，带边界检查
    T& at(std::size_t index) {
        if (index >= size_) {
            throw std::out_of_range("Index out of range in my_vector_raw::at()");
        }
        return data_[index];
    }

    const T& at(std::size_t index) const {
        if (index >= size_) {
            throw std::out_of_range("Index out of range in my_vector_raw::at()");
        }
        return data_[index];
    }
};

//============================================================
// my_vector_smart: 使用智能指针进行内存管理的版本
// 模仿 std::vector 接口，内部使用 std::unique_ptr<T[]> 管理动态数组。
//============================================================

template<typename T>
class my_vector_smart {
private:
    std::unique_ptr<T[]> data_; // 智能指针管理动态数组
    std::size_t size_;          // 当前元素数量
    std::size_t capacity_;      // 当前容量

    // 内部函数：扩展容量到新的容量
    void reserve(std::size_t new_capacity) {
        if (new_capacity <= capacity_) return;
        // 分配新内存
        std::unique_ptr<T[]> new_data = std::make_unique<T[]>(new_capacity);
        // 拷贝现有元素到新内存
        for (std::size_t i = 0; i < size_; ++i) {
            new_data[i] = std::move(data_[i]);
        }
        // 转移新内存到 data_
        data_ = std::move(new_data);
        capacity_ = new_capacity;
    }
```

```
public:
    // (1) 默认构造函数
    my_vector_smart() : data_(nullptr), size_(0), capacity_(0) {}

    // (2) 从初始化列表构造
    my_vector_smart(std::initializer_list<T> init) : data_(nullptr), size_(0), capacity
_(0) {
        reserve(init.size());
        for (const auto& elem : init) {
            push_back(elem);
        }
    }

    // (3) 拷贝构造函数：深拷贝
    my_vector_smart(const my_vector_smart& other) : data_(nullptr), size_(0), capacity_
(0) {
        reserve(other.capacity_);
        for (std::size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
    }

    // (4) 移动构造函数：转移所有权
    my_vector_smart(my_vector_smart&& other) noexcept
    : data_(std::move(other.data_)), size_(other.size_), capacity_(other.capacity_) {
        other.size_ = 0;
        other.capacity_ = 0;
    }

    // (5) 拷贝赋值运算符：深拷贝
    my_vector_smart& operator=(const my_vector_smart& other) {
        if (this == &other) return *this; // 自我赋值检查
        // 清理当前资源
        data_.reset();
        size_ = 0;
        capacity_ = 0;
        // 拷贝新数据
        reserve(other.capacity_);
        for (std::size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
        return *this;
    }

    // (6) 移动赋值运算符：转移所有权
    my_vector_smart& operator=(my_vector_smart&& other) noexcept {
```

```
    if (this == &other) return *this; // 自我赋值检查
    // 释放当前资源并转移
    data_ = std::move(other.data_);
    size_ = other.size_;
    capacity_ = other.capacity_;
    // 重置 other
    other.size_ = 0;
    other.capacity_ = 0;
    return *this;
}

// (7) 析构函数：unique_ptr 自动处理内存释放
~my_vector_smart() = default;

// 添加元素到末尾
void push_back(const T& value) {
    if (size_ == capacity_) {
        // 增加容量，通常是现有容量的两倍，或至少为1
        std::size_t new_capacity = capacity_ == 0 ? 1 : capacity_ * 2;
        reserve(new_capacity);
    }
    data_[size_++] = value;
}

// 移除末尾元素
void pop_back() {
    if (size_ == 0) {
        throw std::out_of_range("pop_back() called on empty my_vector_smart");
    }
    --size_;
    // 可选：调用析构函数（对于非平凡类型）
    // data_[size_].~T();
}

// 返回当前大小
std::size_t size() const noexcept {
    return size_;
}

// 返回当前容量
std::size_t capacity() const noexcept {
    return capacity_;
}

// 检查是否为空
bool empty() const noexcept {
    return size_ == 0;
```

```
    }

    // 下标操作符，无边界检查
    T& operator[](std::size_t index) noexcept {
        return data_[index];
    }

    const T& operator[](std::size_t index) const noexcept {
        return data_[index];
    }

    // at 成员函数，带边界检查
    T& at(std::size_t index) {
        if (index >= size_) {
            throw std::out_of_range("Index out of range in my_vector_smart::at()");
        }
        return data_[index];
    }

    const T& at(std::size_t index) const {
        if (index >= size_) {
            throw std::out_of_range("Index out of range in my_vector_smart::at()");
        }
        return data_[index];
    }
};

//============================================================
// 测试示例
//============================================================

int main() {
    std::cout << "Testing my_vector_raw:\n";
    my_vector_raw<int> vec_raw = {1, 2, 3};
    vec_raw.push_back(4);
    vec_raw.push_back(5);
    for (std::size_t i = 0; i < vec_raw.size(); ++i) {
        std::cout << "vec_raw[" << i << "] = " << vec_raw[i] << "\n";
    }

    // 测试拷贝与移动
    my_vector_raw<int> vec_raw_copy = vec_raw; // 拷贝构造
    my_vector_raw<int> vec_raw_move = std::move(vec_raw); // 移动构造

    std::cout << "\nAfter copying and moving:\n";
    std::cout << "vec_raw_move size: " << vec_raw_move.size() << "\n";
    for (std::size_t i = 0; i < vec_raw_move.size(); ++i) {
```

```
        std::cout << "vec_raw_move[" << i << "] = " << vec_raw_move[i] << "\n";
    }

    std::cout << "\nTesting my_vector_smart:\n";
    my_vector_smart<std::string> vec_smart = {"apple", "banana", "cherry"};
    vec_smart.push_back("date");
    vec_smart.push_back("elderberry");
    for (std::size_t i = 0; i < vec_smart.size(); ++i) {
        std::cout << "vec_smart[" << i << "] = " << vec_smart[i] << "\n";
    }

    // 测试拷贝与移动
    my_vector_smart<std::string> vec_smart_copy = vec_smart; // 拷贝构造
    my_vector_smart<std::string> vec_smart_move = std::move(vec_smart); // 移动构造

    std::cout << "\nAfter copying and moving:\n";
    std::cout << "vec_smart_move size: " << vec_smart_move.size() << "\n";
    for (std::size_t i = 0; i < vec_smart_move.size(); ++i) {
        std::cout << "vec_smart_move[" << i << "] = " << vec_smart_move[i] << "\n";
    }

    return 0;
}
```

3. 模拟实现 std::string，public 继承自 vector<char>

```cpp
#include <iostream>
#include <memory>       // for std::unique_ptr
#include <initializer_list>
#include <utility>      // for std::move
#include <stdexcept>    // for std::out_of_range
#include <cstring>      // for std::strlen, std::strcpy, std::strcmp
#include <algorithm>    // for std::copy

//============================================================
// my_vector_smart: 使用智能指针进行内存管理的版本
// 模仿 std::vector 接口，内部使用 std::unique_ptr<T[]> 管理动态数组。
//============================================================

template<typename T>
class my_vector_smart {
protected:
    std::unique_ptr<T[]> data_; // 智能指针管理动态数组
    std::size_t size_;          // 当前元素数量
    std::size_t capacity_;      // 当前容量

    // 内部函数：扩展容量到新的容量
    void reserve(std::size_t new_capacity) {
        if (new_capacity <= capacity_) return;
        // 分配新内存
        std::unique_ptr<T[]> new_data = std::make_unique<T[]>(new_capacity);
        // 拷贝现有元素到新内存
        for (std::size_t i = 0; i < size_; ++i) {
            new_data[i] = std::move(data_[i]);
        }
        // 转移新内存到 data_
        data_ = std::move(new_data);
        capacity_ = new_capacity;
    }

public:
    // (1) 默认构造函数
    my_vector_smart() : data_(nullptr), size_(0), capacity_(0) {}

    // (2) 从初始化列表构造
    my_vector_smart(std::initializer_list<T> init) : data_(nullptr), size_(0), capacity
_(0) {
        reserve(init.size());
        for (const auto& elem : init) {
            push_back(elem);
        }
    }
```

```
    // (3) 拷贝构造函数：深拷贝
    my_vector_smart(const my_vector_smart& other) : data_(nullptr), size_(0), capacity_
(0) {
        reserve(other.capacity_);
        for (std::size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
    }

    // (4) 移动构造函数：转移所有权
    my_vector_smart(my_vector_smart&& other) noexcept
    : data_(std::move(other.data_)), size_(other.size_), capacity_(other.capacity_) {
        other.size_ = 0;
        other.capacity_ = 0;
    }

    // (5) 拷贝赋值运算符：深拷贝
    my_vector_smart& operator=(const my_vector_smart& other) {
        if (this == &other) return *this; // 自我赋值检查
        // 清理当前资源
        data_.reset();
        size_ = 0;
        capacity_ = 0;
        // 拷贝新数据
        reserve(other.capacity_);
        for (std::size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
        return *this;
    }

    // (6) 移动赋值运算符：转移所有权
    my_vector_smart& operator=(my_vector_smart&& other) noexcept {
        if (this == &other) return *this; // 自我赋值检查
        // 释放当前资源并转移
        data_ = std::move(other.data_);
        size_ = other.size_;
        capacity_ = other.capacity_;
        // 重置 other
        other.size_ = 0;
        other.capacity_ = 0;
        return *this;
    }

    // (7) 析构函数：unique_ptr 自动处理内存释放
    virtual ~my_vector_smart() = default;
```

```
// 添加元素到末尾
void push_back(const T& value) {
    if (size_ == capacity_) {
        // 增加容量，通常是现有容量的两倍，或至少为1
        std::size_t new_capacity = capacity_ == 0 ? 1 : capacity_ * 2;
        reserve(new_capacity);
    }
    data_[size_++] = value;
}

// 移除末尾元素
void pop_back() {
    if (size_ == 0) {
        throw std::out_of_range("pop_back() called on empty my_vector_smart");
    }
    --size_;
    // 可选：调用析构函数（对于非平凡类型）
    // data_[size_].~T();
}

// 返回当前大小
std::size_t size() const noexcept {
    return size_;
}

// 返回当前容量
std::size_t capacity() const noexcept {
    return capacity_;
}

// 检查是否为空
bool empty() const noexcept {
    return size_ == 0;
}

// 下标操作符，无边界检查
T& operator[](std::size_t index) noexcept {
    return data_[index];
}

const T& operator[](std::size_t index) const noexcept {
    return data_[index];
}

// at 成员函数，带边界检查
T& at(std::size_t index) {
```

```
        if (index >= size_) {
            throw std::out_of_range("Index out of range in my_vector_smart::at()");
        }
        return data_[index];
    }

    const T& at(std::size_t index) const {
        if (index >= size_) {
            throw std::out_of_range("Index out of range in my_vector_smart::at()");
        }
        return data_[index];
    }
};

//============================================================
// my_string: 类比 std::string 的自定义字符串类
// 继承自 my_vector_smart<char>，使用智能指针进行内存管理。
// 支持 std::string 的常用功能，包括运算符重载。
//============================================================

class my_string : public my_vector_smart<char> {
public:
    // (1) 默认构造函数
    my_string() : my_vector_smart<char>() {
        // 添加终止符 '\0'
        this->push_back('\0');
        --this->size_; // 不计入终止符
    }

    // (2) 从 C 字符串构造
    my_string(const char* cstr) : my_vector_smart<char>() {
        std::size_t len = std::strlen(cstr);
        this->reserve(len + 1); // 预留空间，包括终止符
        for (std::size_t i = 0; i < len; ++i) {
            this->push_back(cstr[i]);
        }
        this->push_back('\0'); // 添加终止符
        --this->size_; // 不计入终止符
    }

    // (3) 从 std::string 构造
    my_string(const std::string& str) : my_vector_smart<char>() {
        this->reserve(str.size() + 1);
        for (char c : str) {
            this->push_back(c);
        }
        this->push_back('\0');
```

```
    --this->size_;
}

// (4) 拷贝构造函数
my_string(const my_string& other) : my_vector_smart<char>(other) {
    // 确保终止符
    if (this->size_ > 0) {
        this->push_back('\0');
    }
}

// (5) 移动构造函数
my_string(my_string&& other) noexcept : my_vector_smart<char>(std::move(other)) {
    // 确保终止符
    if (this->size_ > 0 && this->operator[](this->size_) != '\0') {
        this->push_back('\0');
        --this->size_;
    }
}

// (6) 拷贝赋值运算符
my_string& operator=(const my_string& other) {
    if (this == &other) return *this; // 自我赋值检查
    my_vector_smart<char>::operator=(other);
    // 确保终止符
    if (this->size_ > 0 && this->operator[](this->size_) != '\0') {
        this->push_back('\0');
        --this->size_;
    }
    return *this;
}

// (7) 移动赋值运算符
my_string& operator=(my_string&& other) noexcept {
    if (this == &other) return *this; // 自我赋值检查
    my_vector_smart<char>::operator=(std::move(other));
    // 确保终止符
    if (this->size_ > 0 && this->operator[](this->size_) != '\0') {
        this->push_back('\0');
        --this->size_;
    }
    return *this;
}

// (8) 析构函数
~my_string() = default;
```

```cpp
// 返回字符串长度（不包括终止符）
std::size_t length() const noexcept {
    return this->size_;
}

// 返回字符串大小（与 length 相同）
std::size_t size_str() const noexcept {
    return this->size_;
}

// 返回是否为空
bool empty_str() const noexcept {
    return this->empty();
}

// 清空字符串
void clear_str() noexcept {
    this->size_ = 0;
    if (this->capacity_ > 0) {
        this->data_[0] = '\0';
        this->push_back('\0');
        --this->size_;
    }
}

// 返回 C 字符串（以 '\0' 结尾）
const char* c_str() const noexcept {
    if (this->size_ == 0) return "";
    // 确保有终止符
    if (this->data_[this->size_] != '\0') {
        // 添加终止符
        const_cast<my_string*>(this)->push_back('\0');
        --const_cast<my_string*>(this)->size_;
    }
    return this->data_.get();
}

// 重载 operator+=，用于字符串拼接
my_string& operator+=(const my_string& other) {
    for (std::size_t i = 0; i < other.size_; ++i) {
        this->push_back(other.data_[i]);
    }
    // 添加终止符
    this->push_back('\0');
    --this->size_;
    return *this;
}
```

```cpp
// 重载operator+=，用于字符拼接
my_string& operator+=(char c) {
    this->push_back(c);
    // 添加终止符
    this->push_back('\0');
    --this->size_;
    return *this;
}

// 重载operator+，用于字符串拼接
friend my_string operator+(const my_string& lhs, const my_string& rhs) {
    my_string result = lhs;
    result += rhs;
    return result;
}

// 重载operator==，用于字符串比较
friend bool operator==(const my_string& lhs, const my_string& rhs) {
    if (lhs.size_ != rhs.size_) return false;
    for (std::size_t i = 0; i < lhs.size_; ++i) {
        if (lhs.data_[i] != rhs.data_[i]) return false;
    }
    return true;
}

// 重载operator!=，用于字符串比较
friend bool operator!=(const my_string& lhs, const my_string& rhs) {
    return !(lhs == rhs);
}

// 重载operator<，用于字符串比较
friend bool operator<(const my_string& lhs, const my_string& rhs) {
    return std::strcmp(lhs.c_str(), rhs.c_str()) < 0;
}

// 重载operator<=，用于字符串比较
friend bool operator<=(const my_string& lhs, const my_string& rhs) {
    return std::strcmp(lhs.c_str(), rhs.c_str()) <= 0;
}

// 重载operator>，用于字符串比较
friend bool operator>(const my_string& lhs, const my_string& rhs) {
    return std::strcmp(lhs.c_str(), rhs.c_str()) > 0;
}
```

```
    // 重载operator>=，用于字符串比较
    friend bool operator>=(const my_string& lhs, const my_string& rhs) {
        return std::strcmp(lhs.c_str(), rhs.c_str()) >= 0;
    }

    // 重载输入流运算符
    friend std::istream& operator>>(std::istream& is, my_string& str) {
        str.clear_str();
        char c;
        while (is.get(c) && !std::isspace(c)) {
            str.push_back(c);
        }
        // 添加终止符
        str.push_back('\0');
        --str.size_;
        return is;
    }

    // 重载输出流运算符
    friend std::ostream& operator<<(std::ostream& os, const my_string& str) {
        os << str.c_str();
        return os;
    }

    // 重载下标操作符，返回引用，允许修改
    char& operator[](std::size_t index) noexcept {
        return this->data_[index];
    }

    const char& operator[](std::size_t index) const noexcept {
        return this->data_[index];
    }

    // at 成员函数，带边界检查
    char& at_str(std::size_t index) {
        return this->at(index);
    }

    const char& at_str(std::size_t index) const {
        return this->at(index);
    }
};

//============================================================
// 测试示例
//============================================================
```

```cpp
int main() {
    // 测试默认构造函数
    my_string s1;
    std::cout << "s1 (默认构造): \"" << s1 << "\"\n";

    // 测试从 C 字符串构造
    my_string s2("Hello");
    std::cout << "s2 (从 C 字符串构造): \"" << s2 << "\"\n";

    // 测试从 std::string 构造
    std::string std_str = "World";
    my_string s3(std_str);
    std::cout << "s3 (从 std::string 构造): \"" << s3 << "\"\n";

    // 测试拷贝构造
    my_string s4 = s2;
    std::cout << "s4 (拷贝 s2): \"" << s4 << "\"\n";

    // 测试移动构造
    my_string s5 = std::move(s3);
    std::cout << "s5 (移动 s3): \"" << s5 << "\"\n";
    std::cout << "s3 (移动后): \"" << s3 << "\"\n";

    // 测试赋值运算符
    my_string s6;
    s6 = s2;
    std::cout << "s6 (赋值 s2): \"" << s6 << "\"\n";

    // 测试移动赋值运算符
    my_string s7;
    s7 = std::move(s4);
    std::cout << "s7 (移动赋值 s4): \"" << s7 << "\"\n";
    std::cout << "s4 (移动后): \"" << s4 << "\"\n";

    // 测试 push_back 和 pop_back
    s1.push_back('A');
    s1.push_back('B');
    s1.push_back('C');
    std::cout << "s1 (添加字符): \"" << s1 << "\"\n";
    s1.pop_back();
    std::cout << "s1 (移除最后一个字符): \"" << s1 << "\"\n";

    // 测试 operator+=
    s2 += s5;
    std::cout << "s2 += s5: \"" << s2 << "\"\n";

    s2 += '!';
```

```
    std::cout << "s2 += '!': \"" << s2 << "\"\n";

    // 测试operator+
    my_string s8 = s2 + s7;
    std::cout << "s8 (s2 + s7): \"" << s8 << "\"\n";

    // 测试比较运算符
    std::cout << "s2 == s5: " << (s2 == s5 ? "true" : "false") << "\n";
    std::cout << "s2 != s5: " << (s2 != s5 ? "true" : "false") << "\n";
    std::cout << "s2 < s7: " << (s2 < s7 ? "true" : "false") << "\n";
    std::cout << "s2 > s7: " << (s2 > s7 ? "true" : "false") << "\n";

    // 测试c_str
    std::cout << "s8 的 C 字符串: \"" << s8.c_str() << "\"\n";

    // 测试输入流运算符
    my_string s9;
    std::cout << "请输入一个单词: ";
    std::cin >> s9;
    std::cout << "你输入的单词是: \"" << s9 << "\"\n";

    return 0;
}
```

4. Complex Number（运算符重载）

```cpp
#include <iostream>
#include <cmath>  // 用于计算绝对值
#include <stdexcept>  // 用于异常处理

class ComplexNumber {
private:
    double real;  // 实部
    double imag;  // 虚部

public:
    // 构造函数
    ComplexNumber(double r = 0.0, double i = 0.0) : real(r), imag(i) {}

    // 输出复数的友元函数
    friend std::ostream& operator<<(std::ostream& os, const ComplexNumber& c) {
        os << c.real << (c.imag >= 0 ? "+" : "") << c.imag << "i";
        return os;
    }

    // 输入复数的友元函数
    friend std::istream& operator>>(std::istream& is, ComplexNumber& c) {
        is >> c.real >> c.imag;
        return is;
    }

    // 加法运算符重载
    ComplexNumber operator+(const ComplexNumber& other) const {
        return ComplexNumber(real + other.real, imag + other.imag);
    }

    // 减法运算符重载
    ComplexNumber operator-(const ComplexNumber& other) const {
        return ComplexNumber(real - other.real, imag - other.imag);
    }

    // 乘法运算符重载
    ComplexNumber operator*(const ComplexNumber& other) const {
        return ComplexNumber(
            real * other.real - imag * other.imag,
            real * other.imag + imag * other.real
            );
    }

    // 除法运算符重载
    ComplexNumber operator/(const ComplexNumber& other) const {
        double denominator = other.real * other.real + other.imag * other.imag;
```

```
    if (denominator == 0) {
        throw std::invalid_argument("Attempt to divide by zero");
    }
    return ComplexNumber(
        (real * other.real + imag * other.imag) / denominator,
        (imag * other.real - real * other.imag) / denominator
        );
}

// 赋值运算符重载
ComplexNumber& operator=(const ComplexNumber& other) {
    if (this != &other) {  // 防止自赋值
        real = other.real;
        imag = other.imag;
    }
    return *this;
}

// 比较运算符重载 (相等)
bool operator==(const ComplexNumber& other) const {
    return real == other.real && imag == other.imag;
}

// 比较运算符重载 (不等)
bool operator!=(const ComplexNumber& other) const {
    return !(*this == other);
}

// 大于运算符重载 (按模长大小比较)
bool operator>(const ComplexNumber& other) const {
    return magnitude() > other.magnitude();
}

// 小于运算符重载 (按模长大小比较)
bool operator<(const ComplexNumber& other) const {
    return magnitude() < other.magnitude();
}

// 前置自增运算符重载
ComplexNumber& operator++() {
    real++;
    imag++;
    return *this;
}

// 后置自增运算符重载
ComplexNumber operator++(int) {
```

```
        ComplexNumber temp = *this;
        ++(*this);
        return temp;
    }

    // 计算复数的模长
    double magnitude() const {
        return std::sqrt(real * real + imag * imag);
    }

    // 获取实部
    double getReal() const { return real; }

    // 获取虚部
    double getImag() const { return imag; }
};

int main() {
    ComplexNumber c1(3, 4);  // 创建复数 3 + 4i
    ComplexNumber c2(1, 2);  // 创建复数 1 + 2i

    // 加法
    ComplexNumber c3 = c1 + c2;
    std::cout << "c1 + c2 = " << c3 << std::endl;  // 输出: c1 + c2 = 4+6i

    // 乘法
    ComplexNumber c4 = c1 * c2;
    std::cout << "c1 * c2 = " << c4 << std::endl;  // 输出: c1 * c2 = -5+10i

    // 除法
    try {
        ComplexNumber c5 = c1 / c2;
        std::cout << "c1 / c2 = " << c5 << std::endl;  // 输出: c1 / c2 = 2.2-0.4i
    } catch (const std::invalid_argument& e) {
        std::cerr << e.what() << std::endl;
    }

    // 赋值
    ComplexNumber c6;
    c6 = c1;
    std::cout << "c6 = " << c6 << std::endl;  // 输出: c6 = 3+4i

    // 比较运算符
    if (c1 == c2) {
        std::cout << "c1 and c2 are equal." << std::endl;
    } else {
```

```
        std::cout << "c1 and c2 are not equal." << std::endl;  // 输
出: c1 and c2 are not equal.
    }

    // 自增运算符
    std::cout << "++c1 = " << ++c1 << std::endl;  // 输出: ++c1 = 4+5i

    // 后置自增运算符
    std::cout << "c1++ = " << c1++ << std::endl;  // 输出: c1++ = 4+5i
    std::cout << "After c1++, c1 = " << c1 << std::endl;  // 输
出: After c1++, c1 = 5+6i

    // 计算模长
    std::cout << "Magnitude of c1: " << c1.magnitude() << std::endl;  // 输
出: Magnitude of c1: 7.81025

    return 0;
}
```

5. 函数调用运算符的重载

```cpp
#include <iostream>

class Multiplier {
private:
    int factor;  // 内部状态

public:
    // 构造函数，设置初始因子
    Multiplier(int f) : factor(f) {}

    // 重载函数调用运算符
    int operator()(int x) {
        return x * factor;  // 将输入值乘以 factor
    }
};

int main() {
    Multiplier triple(3);  // 创建一个因子为 3 的函数对象
    std::cout << "Triple of 5: " << triple(5) << std::endl;  // 输出 15

    Multiplier doubleIt(2);  // 创建一个因子为 2 的函数对象
    std::cout << "Double of 7: " << doubleIt(7) << std::endl;  // 输出 14

    return 0;
}
```

6. Basic STL Exercises

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <numeric>

/**
 * @brief Given a vector of strings @c strings, reduce this vector so that each
 * string appears only once. After the function call, @c strings contains the
 * same set of elements (possibly sorted) as the original vector, but each
 * element appears only once.
 *
 * @param strings The given vector of strings.
 */
void dropDuplicates(std::vector<std::string>& strings) {
    // Sort the strings to bring duplicates together
    std::sort(strings.begin(), strings.end());
    // Remove duplicates
    auto last = std::unique(strings.begin(), strings.end());
    // Erase the duplicates from the vector
    strings.erase(last, strings.end());
}

/**
 * @brief Given a vector of strings @c strings and a nonnegative integer @c k,
 * partition the vector into two parts (with the elements rearranged) so that
 * the first part contains all the strings with length no greater than @c k and
 * the second part contains the rest of the strings. Return an iterator just
 * past the last element of the first part.
 *
 * @param strings The given vector of strings.
 * @return An iterator just past the last element of the first part. In
 * particular, if the second part is not empty, the returned iterator will refer
 * to the first element of the second part.
 */
auto partitionByLength(std::vector<std::string>& strings, std::size_t k) {
    return std::partition(strings.begin(), strings.end(),
        [k](const std::string& s) {
            return s.length() <= k;
        });
}

/**
 * @brief Generate all the permutations of {1, 2, ..., n} in lexicographical
 * order, and print them to @c os.
 *
```

```
 */
void generatePermutations(int n, std::ostream &os) {
    std::vector<int> numbers(n);
    std::iota(numbers.begin(), numbers.end(), 1);
    do {
        // DONE: Print the numbers, separated by a space.
        std::for_each(numbers.begin(), numbers.end(),
            [&os](int num) {
                os << num << ' ';
            });
        os << '\n';
    } while (std::next_permutation(numbers.begin(), numbers.end()));
}
```

7. Snake

utils.h:

```
#ifndef SNAKE_UTILS_H
#define SNAKE_UTILS_H

#if defined(__MINGW64_VERSION_MAJOR) && __MINGW64_VERSION_MAJOR < 7
    #error Your MinGW version is too old. Go to winlibs.com to install the latest MinGW-
w64.
#endif

#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#ifdef _WIN32
#include <conio.h>
#include <windows.h>
#else
#include <sys/ioctl.h>
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>

// Test whether the keyboard has been stroke. Returns 1 if there is a keyboard
// stroke, or 0 if there is none.
int kbhit(void) {
    struct termios oldt;
    tcgetattr(STDIN_FILENO, &oldt);
    struct termios newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    int oldf = fcntl(STDIN_FILENO, F_GETFL, 0);
    fcntl(STDIN_FILENO, F_SETFL, oldf | O_NONBLOCK);

    int ch = getchar();

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    fcntl(STDIN_FILENO, F_SETFL, oldf);

    if (ch != EOF) {
        ungetc(ch, stdin);
```

```
        return 1;
    }
    return 0;
}

// Get a character from input without showing it.
int getch(void) {
    struct termios oldt;
    tcgetattr(STDIN_FILENO, &oldt);
    struct termios newt = oldt;
    newt.c_lflag &= ~ICANON;
    newt.c_lflag &= ~ECHO;
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    int ch = getchar();
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return ch;
}
#endif // _WIN32

void hide_cursor(void) {
    wprintf(L"\033[?25l");
}

void show_cursor(void) {
    wprintf(L"\033[?25h");
}

void disable_input_echo(void) {
#ifdef _WIN32
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    DWORD mode = 0;
    GetConsoleMode(hStdin, &mode);
    SetConsoleMode(hStdin, mode & ~ENABLE_ECHO_INPUT);
#else
    struct termios t;
    tcgetattr(STDIN_FILENO, &t);
    t.c_lflag &= ~ECHO;
    tcsetattr(STDIN_FILENO, TCSANOW, &t);
#endif // _WIN32
}

void enable_input_echo(void) {
#ifdef _WIN32
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    DWORD mode = 0;
    GetConsoleMode(hStdin, &mode);
    SetConsoleMode(hStdin, mode | ENABLE_ECHO_INPUT);
```

```
#else
    struct termios t;
    tcgetattr(STDIN_FILENO, &t);
    t.c_lflag |= ECHO;
    tcsetattr(STDIN_FILENO, TCSANOW, &t);
#endif // _WIN32
}

bool stdout_is_terminal(void) {
#ifdef _WIN32
    return GetFileType(GetStdHandle(STD_OUTPUT_HANDLE)) == FILE_TYPE_CHAR;
#else
    return isatty(STDOUT_FILENO);
#endif // _WIN32
}

volatile sig_atomic_t sig_status = 0;

void sigint_handler(int sig) {
    sig_status = sig;
}

void exit_on_sigint(void) {
    if (sig_status == SIGINT)
        exit(SIGINT);
}

int is_terminal_size_small(int prows, int pcols) {
#ifdef _WIN32
    // check the terminal size
    CONSOLE_SCREEN_BUFFER_INFO csbi;
    HANDLE hStd = GetStdHandle(STD_OUTPUT_HANDLE);
    if (!GetConsoleScreenBufferInfo(hStd, &csbi)) {
        exit(EXIT_FAILURE);
    }
    if (csbi.srWindow.Bottom < prows + 3 || csbi.srWindow.Right < pcols * 2) {
        wprintf(L"Terminal size is small than %d * %d. Please enlarge the terminal.\n", pr
ows + 3, pcols * 2);
        return 1;
    }
#else
    // check the terminal size
    struct winsize w;
    ioctl(STDOUT_FILENO, TIOCGWINSZ, &w);
    if (w.ws_row < prows + 3 || w.ws_col < pcols * 2) {
        wprintf(L"Terminal size is small than %d * %d. Please enlarge the terminal.\n", pr
ows + 3, pcols * 2);
```

```
        return 1;
    }
#endif // _WIN32
    return 0;
}

void prepareGame(void) {
    if (!stdout_is_terminal()) {
        fputs("game must be run in a terminal.", stderr);
        exit(EXIT_FAILURE);
    }

#ifdef _WIN32
    HANDLE hStd = GetStdHandle(STD_OUTPUT_HANDLE);
    if (hStd == INVALID_HANDLE_VALUE) {
        exit(EXIT_FAILURE);
    }
    DWORD dwMode = 0;
    if (!GetConsoleMode(hStd, &dwMode)) {
        exit(EXIT_FAILURE);
    }
    dwMode |= ENABLE_VIRTUAL_TERMINAL_PROCESSING;
    if (!SetConsoleMode(hStd, dwMode)) {
        fputs("Can't enable virtual terminal processing.", stderr);
        exit(EXIT_FAILURE);
    }
#endif // _WIN32

    hide_cursor();
    atexit(show_cursor);

    signal(SIGINT, sigint_handler);

    disable_input_echo();
    // setbuf(stdout, NULL);
}

void move_cursor(int row, int col) {
    wprintf(L"%c[%d;%df", 0x1B, row + 1, col + 1);
}

void clear_screen(void) {
    wprintf(L"\033[2J\033[1;1f");
}

#define ANSI_ESCAPE_RED "31"
#define ANSI_ESCAPE_GREEN "32"
```

```c
#define ANSI_ESCAPE_YELLOW "33"
#define ANSI_ESCAPE_BLUE "34"
#define ANSI_ESCAPE_BRIGHT_YELLOW "93"
#define ANSI_ESCAPE_BRIGHT_BLUE "94"
#define ANSI_ESCAPE_NORMAL "0"

#define SET_COLOR(color) "\033[" ANSI_ESCAPE_##color "m"

#define COLORED(color, text) (SET_COLOR(color) text SET_COLOR(NORMAL))

#define RED_TEXT(text) COLORED(RED, text)
#define GREEN_TEXT(text) COLORED(GREEN, text)
#define YELLOW_TEXT(text) COLORED(YELLOW, text)
#define BLUE_TEXT(text) COLORED(BLUE, text)
#define BRIGHT_YELLOW_TEXT(text) COLORED(BRIGHT_YELLOW, text)
#define BRIGHT_BLUE_TEXT(text) COLORED(BRIGHT_BLUE, text)

void to_next_line(FILE *stream) {
    int ch = fgetc(stream);
    while (ch != EOF && ch != '\n')
        ch = fgetc(stream);
}

void read_line(FILE *stream, char *line) {
    int ch = fgetc(stream);
    while (ch != EOF && ch != '\n') {
        *line++ = (char)ch;
        ch = fgetc(stream);
    }
    *line = '\0';
}

#ifdef _WIN32
#include <windows.h>
void sleep_seconds(int seconds) {
    Sleep(seconds * 1000); // Windows: Use Sleep, parameter is milliseconds
}
#else
#include <unistd.h>
void sleep_seconds(int seconds) {
    sleep(seconds); // Linux/Unix: Use sleep, parameter is seconds
}
#endif

#endif // SNAKE_UTILS_H
```

snake.h:

```c
#ifndef SNAKE_H
#define SNAKE_H

#include <assert.h>
#include <ctype.h>
#include <errno.h>
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <wchar.h>
#include <locale.h>
#include "utils.h"

#define MAX_FOODS 3
#define SEC_PER_SNAKE_MOVE 0.4
#define WIN_SCORE 10
#define NUM_ROWS 20
#define NUM_COLS 20
#define INIT_SNAKE_LEN 3
#define WALL_STR L"□ "
#define BODY_STR L"● "
#define FOOD_STR L"■ "
#define AIR_STR L"  "
#define HEAD_STR L"○ "

typedef struct {
    int row;
    int col;
} Coord;

typedef enum {
    Up,
    Left,
    Right,
    Down
} Direction;

typedef struct SnakeNode {
    Coord pos;
    struct SnakeNode *next;
} SnakeNode;
```

```c
typedef enum {
    GS_Preparing,
    GS_Running,
    GS_Win,
    GS_Lose
} GameStatus;

typedef enum {
    DR_None,
    DR_HitWall,
    DR_HitSelf
} DeathReason;

typedef struct {
    Coord pos;
} Food;

typedef struct {
    GameStatus status;
    int score;
    int numRows;
    int numCols;
    SnakeNode *pSnakeHead;
    Food foods[MAX_FOODS];
    Direction currentDirection;
    DeathReason deathReason;
} Game;


Game createGame(void);
void printInitialGame(const Game *pGame);
void runGame(Game *pGame);
bool gameEnds(Game *pGame);
void destroyGame(Game *pGame);
SnakeNode *createSnake(Game *pGame);
void destroySnake(SnakeNode *pHead);
bool isFoodOnFoods(const Food foods[], int numFoods, const Food food);
Food createFood(const SnakeNode *pSnakeHead, const Food foods[], int numFoods);
SnakeNode *createNewSnakeHead(const Game *pGame);
bool isFoodOnSnake(const SnakeNode *pSnakeHead, const Food food);
bool snakeHitWall(const Game *pGame, SnakeNode *pNewHead);
bool snakeHitSelf(const Game *pGame, SnakeNode *pNewHead);
bool snakeDie(Game *pGame, SnakeNode *pNewHead);
bool snakeEatFood(Game *pGame, SnakeNode *pNewHead);
void snakeMoveNormal(Game *pGame, SnakeNode *pNewHead);
void getSnakeMovement(Game *pGame);
```

```c
bool isOppositeDirection(Direction dir1, Direction dir2);
#endif
```

snake.c:

```c
#include "snake.h"

/**
 * @brief Set the number of score, rows, columns and the initial direction.
 *
 * @param status The status of the game.
 * @param score The score of the game.
 * @param numRows The number of rows in the game.
 * @param numCols The number of columns in the game.
 * @param currentDirection The direction of snake at the current frame.
 * @return The created Game object.
 */
Game createGame(void) {
    Game game = {
        .status = GS_Preparing,
        .score = 0,
        .numRows = NUM_ROWS,
        .numCols = NUM_COLS,
        .currentDirection = Up,
        .deathReason = DR_None,
    };

    // Create snakehead
    game.pSnakeHead = createSnake(&game);

    // Create food
    // Updated: avoid food overlapping
    for (int i = 0; i < MAX_FOODS; i++) {
        game.foods[i] = createFood(game.pSnakeHead, game.foods, i);
    }

    return game;
}


void printInitialGame(const Game *pGame) {
    // Clear the screen first.
    clear_screen();

    // Draw the walls.
    for (int i = 0; i < pGame->numRows; i++) {
```

```
        for (int j = 0; j < pGame->numCols; j++) {
            if (i == 0 || i == pGame->numRows - 1 || j == 0 || j == pGame->numCols - 1) {
                wprintf(WALL_STR);
            } else {
                wprintf(AIR_STR);
            }
        }
        wprintf(L"\n");
    }

    // Draw the snake.
    SnakeNode *pcur = pGame->pSnakeHead;
    move_cursor(pcur->pos.row, 2 * pcur->pos.col);
    wprintf(HEAD_STR);
    pcur = pcur->next;
    while (pcur != NULL) {
        move_cursor(pcur->pos.row, 2 * pcur->pos.col);
        wprintf(BODY_STR);
        pcur = pcur->next;
    }

    // Draw the food
    for (int i = 0; i < MAX_FOODS; i++) {
        move_cursor(pGame->foods[i].pos.row, 2 * pGame->foods[i].pos.col);
        wprintf(FOOD_STR);
    }

    // Print the prompt
    move_cursor(pGame->numRows + 1, 0);
    wprintf(L"Snake needs food! (control by w/a/s/d)\n");

    move_cursor(pGame->numRows + 2, 0);
    wprintf(L"Press CAPSLOCK to avoid interference from input methods\n");

    // Print the score
    move_cursor(pGame->numRows + 3, 0);
    wprintf(L"Score: %d", pGame->score);
}

/**
 * @brief Create a snake with INIT_SNAKE_LEN length.
 * @param pGame The pointer that points to the object Game.
 * @return The head of the snake, which is a pointer of type SnakeNode.
 */
SnakeNode *createSnake(Game *pGame) {
    int midRow = pGame->numRows / 2;
    int midCol = pGame->numCols / 2;
```

```
    SnakeNode *head = malloc(sizeof(SnakeNode));
    if (head == NULL) {
        perror("Failed to allocate memory for snake head");
        exit(EXIT_FAILURE);
    }
    head->pos.row = midRow;
    head->pos.col = midCol;
    head->next = NULL;

    SnakeNode *current = head;
    for (int i = 1; i < INIT_SNAKE_LEN; i++) {
        SnakeNode *node = malloc(sizeof(SnakeNode));
        if (node == NULL) {
            perror("Failed to allocate memory for snake node");
            exit(EXIT_FAILURE);
        }
        node->pos.row = midRow + i;
        node->pos.col = midCol;
        node->next = NULL;
        current->next = node;
        current = node;
    }
    return head;
}


/**
 * @brief Check if the newly generated food overlaps with existing food.
 * @param foods Array of existing food items.
 * @param numFoods The number of food items.
 * @param food The newly generated food item.
 * @return Returns true if the new food overlaps with existing food; otherwise, returns fa
lse.
 */
bool isFoodOnFoods(const Food foods[], int numFoods, const Food food) {
    for (int i = 0; i < numFoods; i++) {
        if (foods[i].pos.row == food.pos.row && foods[i].pos.col == food.pos.col) {
            return true;
        }
    }
    return false;
}

/**
 * @brief Create a new food item at a random position not overlapping with the snake or ex
isting foods.
```

```
 * @param pSnakeHead Pointer to the head of the snake.
 * @param foods Array of existing food items.
 * @param numFoods The number of existing food items.
 * @return Returns a new Food item with a valid position.
 */

Food createFood(const SnakeNode *pSnakeHead, const Food foods[], int numFoods) {
    Food food;
    do {
        food.pos.row = rand() % (NUM_ROWS - 2) + 1;
        food.pos.col = rand() % (NUM_COLS - 2) + 1;
    } while (isFoodOnSnake(pSnakeHead, food) || isFoodOnFoods(foods, numFoods, food));
    return food;
}



/**
 * @brief Get the input from the keyboard; change the moving direction of
 * the snake.
 * @param pGame The pointer to the object Game.
 * Update: Uppercase & lowercase supported
 * Update: prevent immediate death in the opposite direction
 */
void getSnakeMovement(Game *pGame) {
    if (kbhit()) {
        char ch = getch();
        Direction newDirection = pGame->currentDirection;
        switch (ch) {
            case 'w':
            case 'W': // recommend to press CAPSLOCK when playing the game to avoid the in
terference of Chinese imput method
                newDirection = Up;
                break;
            case 's':
            case 'S':
                newDirection = Down;
                break;
            case 'a':
            case 'A':
                newDirection = Left;
                break;
            case 'd':
            case 'D':
                newDirection = Right;
                break;
            default:
                break;
```

```
        }

        // Update: If the new direction is not the opposite of the current direction, upda
te the current direction
        if (!isOppositeDirection(pGame->currentDirection, newDirection)) {
            pGame->currentDirection = newDirection;
        }
    }
}


/**
 * @brief create a new invisible snakehead according to the current direction.
 * @return A pointer to SnakeNode that represents the new snake head.
 */
SnakeNode *createNewSnakeHead(const Game *pGame) {
    SnakeNode *newHead = malloc(sizeof(SnakeNode));
    if (newHead == NULL) {
        perror("Failed to allocate memory for new snake head");
        exit(EXIT_FAILURE);
    }

    // Get current head position
    int row = pGame->pSnakeHead->pos.row;
    int col = pGame->pSnakeHead->pos.col;

    // Move into the next direction
    switch (pGame->currentDirection) {
        case Up:
            row -= 1;
            break;
        case Down:
            row += 1;
            break;
        case Left:
            col -= 1;
            break;
        case Right:
            col += 1;
            break;
    }

    // Saving changes
    newHead->pos.row = row;
    newHead->pos.col = col;
    newHead->next = NULL;
```

```
    return newHead;
}

/**
 * @brief Check if the snake hits the wall.
 * @return True if the new head is out of the wall.
 */
bool snakeHitWall(const Game *pGame, SnakeNode *pNewHead) {
    int row = pNewHead->pos.row;
    int col = pNewHead->pos.col; //current position

    return (row <= 0 || row >= (pGame->numRows) - 1 || col <= 0 || col >= (pGame->numCols)
 - 1);
}

/**
 * @brief Check if the snake hits itself.
 * @return True if the new head hit the snake itself.
 */
bool snakeHitSelf(const Game *pGame, SnakeNode *pNewHead) {
    SnakeNode *current = pGame->pSnakeHead;
    while (current != NULL) {
        if (current->pos.row == pNewHead->pos.row && current->pos.col == pNewHead->pos.col
) {
            return true;
        }
        current = current->next;
    }
    return false;
}

/**
 * @brief Check if the snake crushed on obstacles. If so, the status of the
 * game will set to lose (handled in the two functions).
 * @return True if the new head crushed on obstacles.
 * Update: Death Reason recorded and updated ('const' *pGame removed)
 */
bool snakeDie(Game *pGame, SnakeNode *pNewHead) {
    if (snakeHitWall(pGame, pNewHead)) {
        pGame->deathReason = DR_HitWall;
        return true;
    } else if (snakeHitSelf(pGame, pNewHead)) {
        pGame->deathReason = DR_HitSelf;
        return true;
    } else {
        pGame->deathReason = DR_None; // Default deathReason for not died case
        return false;
```

```
    }
}
/**
 * @brief Check if the snake eats the food. If so, make a new food and make the head appea
r and append the new head,
 * then increase the score.
 * @return True if the snake eats the food.
 */
bool snakeEatFood(Game *pGame, SnakeNode *pNewHead) {
    for (int i = 0; i < MAX_FOODS; i++) {
        if (pNewHead->pos.row == pGame->foods[i].pos.row && pNewHead->pos.col == pGame->fo
ods[i].pos.col) {
            // Create new snakehead
            pNewHead->next = pGame->pSnakeHead;
            pGame->pSnakeHead = pNewHead;

            // Display new snakehead
            move_cursor(pNewHead->pos.row, 2 * pNewHead->pos.col);
            wprintf(HEAD_STR);

            // Display the body at the position of the old snake head
            move_cursor(pNewHead->next->pos.row, 2 * pNewHead->next->pos.col);
            wprintf(BODY_STR);

            // Generate new food to avoid overlapping with existing food
            pGame->foods[i] = createFood(pGame->pSnakeHead, pGame->foods, MAX_FOODS);

            // Display new food
            move_cursor(pGame->foods[i].pos.row, 2 * pGame->foods[i].pos.col);
            wprintf(FOOD_STR);

            // Add score
            pGame->score += 1;

            // Update current score display
            move_cursor(pGame->numRows + 3, 0);
            wprintf(L"Score: %d  ", pGame->score); // Add whitespace to overwrite the prev
ious ones

            return true;
        }
    }
    return false;
}


/**
```

```
 * @brief Check if the food is on the snake.
 * @return True if the food is on the snake, false otherwise.
 */
bool isFoodOnSnake(const SnakeNode *pSnakeHead, const Food food) {
    const SnakeNode *current = pSnakeHead;
    while (current != NULL) {
        if (current->pos.row == food.pos.row && current->pos.col == food.pos.col) {
            return true;
        }
        current = current->next;
    }
    return false;
}

/**
 * @brief Move the snake by one step.
 *
 */
void snakeMoveNormal(Game *pGame, SnakeNode *pNewHead) {
    // Append new head
    pNewHead->next = pGame->pSnakeHead;
    pGame->pSnakeHead = pNewHead;

    // Display new head
    move_cursor(pNewHead->pos.row, 2 * pNewHead->pos.col);
    wprintf(HEAD_STR);

    // Display BODY_STR at old head's position
    move_cursor(pNewHead->next->pos.row, 2 * pNewHead->next->pos.col);
    wprintf(BODY_STR);

    // Find the tail
    SnakeNode *prev = NULL;
    SnakeNode *current = pGame->pSnakeHead;
    while (current->next != NULL) {
        prev = current;
        current = current->next;
    }

    // current is the tail node
    // Hide the tail
    move_cursor(current->pos.row, 2 * current->pos.col);
    wprintf(AIR_STR);

    // Free the tail node
    free(current);
```

```
    // Set prev->next to NULL
    if (prev != NULL) {
        prev->next = NULL;
    }
}

/**
 * @brief Destroy the snake, free all the nodes of the snake.
 */
void destroySnake(SnakeNode *pHead) { // From "head"
    SnakeNode *current = pHead;
    while (current != NULL) { // Exit after reach the "end+1" position
        SnakeNode *next = current->next;
        free(current);
        current = next;
    }
}

/**
 * @brief Destroy the game.
 */
void destroyGame(Game *pGame) {
    destroySnake(pGame->pSnakeHead);
}

bool gameEnds(Game *pGame) {
    if (pGame->status == GS_Lose) {
        move_cursor(pGame->numRows + 3, 0);
        switch (pGame->deathReason) {
            case DR_HitWall:
                wprintf(L"Game Over! You hit the wall! Your score is %d\n", pGame->score);
                break;
            case DR_HitSelf:
                wprintf(L"Game Over! You ran into yourself! Your score is %d\n", pGame->sc
ore);
                break;
            default:
                wprintf(L"Game Over! Your score is %d\n", pGame->score);
                break;
        }
        sleep_seconds(2);
        return true;
    }
    if (pGame->score == WIN_SCORE) {
        pGame->status = GS_Win;
        move_cursor(pGame->numRows + 3, 0);
        wprintf(L"Congratulations! You win 10pts!\n");
```

```
        sleep_seconds(2); // (Optional)
        // Choose one method (here or in line 471) to display results to users before exit
ing
        return true;
    }
    return false;
}


void runGame(Game *pGame) {
    // Check if the terminal size is too small.
    if (is_terminal_size_small(NUM_ROWS, NUM_COLS)) {
        return;
    }
    // Print the frame and initial snake and food.
    printInitialGame(pGame);
    pGame->status = GS_Running;
    clock_t lastSnakeMoveTime = clock();
    while (1) {
        exit_on_sigint();
        if (gameEnds(pGame))
            break;
        getSnakeMovement(pGame);
        clock_t currentTime = clock();
        if ((double)(currentTime - lastSnakeMoveTime) / CLOCKS_PER_SEC >=
            SEC_PER_SNAKE_MOVE) {
            lastSnakeMoveTime = currentTime;
            if (gameEnds(pGame))
                break;

            // Create new snakehead
            SnakeNode *pNewHead = createNewSnakeHead(pGame);

            if (snakeDie(pGame, pNewHead)) {
                // Set failure status. Reason recoreded in "snakeDie" function.
                pGame->status = GS_Lose;
                free(pNewHead);
            } else if (!snakeEatFood(pGame, pNewHead)) {
                snakeMoveNormal(pGame, pNewHead);
            }
        }
    }
}

/**
 * @brief Check if two directions are opposite.
 * @param dir1 The first direction.
```

```
 * @param dir2 The second direction.
 * @return Returns true if the two directions are opposite; otherwise, returns false.
 */
bool isOppositeDirection(Direction dir1, Direction dir2) {
    return (dir1 == Up && dir2 == Down) ||
           (dir1 == Down && dir2 == Up) ||
           (dir1 == Left && dir2 == Right) ||
           (dir1 == Right && dir2 == Left);
}

int main(void) {
    setlocale(LC_ALL, "");
    srand((unsigned int)time(NULL));
    prepareGame();
    Game game = createGame();
    runGame(&game);
    destroyGame(&game);
    // (Optional) system("pause");
    return 0;
}
```

8. 判断是否为回文串

```c
// solution.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

bool is_palindrome(const char *s, int len) {
    for (int i = 0; i < len / 2; ++i) {
        if (s[i] != s[len - 1 - i])
            return 0;
    }
    return 1;
}

int main() {
    int n;
    scanf("%d", &n);
    while (n--) {
        size_t len;
        scanf("%zu", &len);
        getchar();

        char *s = (char *)calloc(len + 1, sizeof(char));
        if (s == NULL)
            return 1;

        size_t num_read = fread(s, 1, len, stdin);
        if (num_read != len) {
            free(s);
            return 1;
        }

        char c = getchar();
        if (c != '\n' && c != EOF) {
            while ((c = getchar()) != '\n' && c != EOF);
        }

        puts(is_palindrome(s, len) ? "Yes" : "No");
        free(s);
    }
    return 0;
}
```

9. 手动实现 C 风格字符处理

```c
#include <stddef.h>

int hw3_islower(int ch) {
    return (ch >= 'a' && ch <= 'z');
}

int hw3_isupper(int ch) {
    return (ch >= 'A' && ch <= 'Z');
}

int hw3_isalpha(int ch) {
    return ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'));
}

int hw3_isdigit(int ch) {
    return (ch >= '0' && ch <= '9');
}

int hw3_tolower(int ch) {
    return (ch >= 'A' && ch <= 'Z') ? (ch + ('a' - 'A')) : ch;
}

int hw3_toupper(int ch) {
    return (ch >= 'a' && ch <= 'z') ? (ch - ('a' - 'A')) : ch;
}

size_t hw3_strlen(const char *str) {
    size_t len = 0;
    while (str[len] != '\0')
        ++len;
    return len;
}

char* hw3_strchr(const char *str, int ch) {
    while (*str != '\0') {
        if (*str == (char)ch) {
            return (char *)str;
        }
        str++;
    }
    if (ch == '\0') {
        return (char *)str;
    }
    return NULL;
}
```

```c
char* hw3_strcpy(char *dest, const char *src) {
    char *d = dest;
    while ((*d++ = *src++) != '\0') {
        ; //nothing
    }
    return dest;
}

char* hw3_strcat(char *dest, const char *src) {
    char *d = dest;
    while (*d != '\0') {
        d++;
    }
    while ((*d++ = *src++) != '\0') {
        ; //nothing
    }
    return dest;
}

int hw3_strcmp(const char *lhs, const char *rhs) {
    while (*lhs != '\0' && *lhs == *rhs) {
        lhs++;
        rhs++;
    }
    return (unsigned char) * lhs - (unsigned char) * rhs;
}
```